DATABRICKS CERTIFIED DATA ENGINEER

Advanced Track — Chapter 6

Storing Data Securely:

Governance, Privacy, PII Protection & Change Data Feed (CDF)

A Complete Revision & Study Guide

Prepared for: Parth

Based on your Chapter 6 course slides, expanded with plain-English explanations, real-world analogies, comparison tables, and an exam-ready glossary & cheat sheet.

How to Use This Guide

This guide rebuilds your Chapter 6 slide deck into a structured revision document. Every original topic is preserved, but each one is expanded with a plain-English explanation, an everyday analogy where it helps, and — for the harder concepts — a worked example. New terms (like Terraform) are explained the first time they appear and are also collected in the Glossary at the end.

- **Part 1–2:** Regulatory compliance (GDPR/CCPA) and the foundations of Unity Catalog governance.
- **Part 3–4:** Auditing with System Tables, and how Unity Catalog physically isolates and secures your data.
- **Part 5:** PII protection techniques — pseudonymization vs. anonymization.
- **Part 6–7:** Streaming data challenges, Change Data Feed (CDF), and how deletion/GDPR requests are actually implemented.
- **Part 8:** A full A–Z glossary of every important term in this chapter (including Terraform).
- **Part 9:** A one-page-style cheat sheet of comparison tables and commands for last-minute revision.
- **Part 10:** 15 self-check practice questions with an answer key.

> **Exam Tip**
> This chapter is scenario-heavy on the real exam. Expect questions phrased as "a compliance team needs to delete a user's data" or "an analyst should see hashed SSNs" — you'll be tested on picking the right *tool* (dynamic view vs. row filter vs. CDF vs. VACUUM) for the scenario, not just definitions.

Chapter 6 Roadmap

Chapter 6 sits under the umbrella topic "Governing and Securing Data" and is built from four lecture blocks. Here's the big picture before we dive into details:

Lecture Block	Core Question It Answers	Key Tools/Features
Storing Data Securely	What do regulations require, and how do we think about privacy?	GDPR, CCPA, Identify-Protect-Manage framework
Unity Catalog (Governance, Audit, Isolation)	How does Databricks actually implement access control?	ACLs, Dynamic Views, Row Filters/Column Masks, System Tables, Metastore/Catalog/Schema hierarchy
PII Data Security	How do we make sensitive data safe to use?	Pseudonymization (hashing, tokenization), Anonymization (suppression, generalization)
Streaming Data & CDF	How do we handle updates/deletes in an append-only streaming world, and honor a GDPR deletion request?	Change Data Feed (CDF), table_changes(), VACUUM, DML on streaming tables

Part 1 — Regulatory Compliance & Data Privacy Fundamentals

1.1 Why Compliance Matters: GDPR & CCPA

Even if your team never directly touches personally identifiable information (PII), your company almost certainly does — and that means your data platform is in scope for privacy law. The two big regulations referenced throughout this chapter are:

- **GDPR (General Data Protection Regulation):** The EU's data protection law. If it applies to you, you generally must respond to a data subject's request (to see, export, correct, or delete their data) within 30 days. Non-compliance can cost up to 4% of annual global revenue or €20 million — whichever is larger.
- **CCPA (California Consumer Privacy Act):** California's equivalent. You must acknowledge a request within 10 business days and resolve it within 45 days. Penalties run up to $2,500 per violation, or $7,500 per intentional violation, plus statutory damages per consumer per incident.

Because most global companies do business in both regions, Databricks encourages you to design one policy that satisfies the stricter of the two rather than maintaining separate compliance tracks — this is simply less operational overhead.

> **In Plain English**
> Think of GDPR/CCPA like a landlord-tenant law: the tenant (the customer) has the right to ask "show me what you have on me," "fix this," or "delete all of it" — and the landlord (your company) has a legal clock ticking the moment that request lands. Your data platform needs to be built so that clock is easy to beat, not scary.

1.2 How Databricks Simplifies Compliance

The slide deck highlights that the Lakehouse architecture (Delta Lake + Databricks platform) is specifically well-suited to compliance work, for a few concrete reasons:

- **Fewer copies of PII:** Because Delta Lake lets huge volumes of data live in one well-governed place and still be queried fast, you don't need to copy sensitive data into five different downstream systems just for performance. Fewer copies = fewer places a "right to be forgotten" request has to chase down.
- **Find personal data quickly:** Unity Catalog's search, tagging and lineage (covered in Part 2) mean you can actually answer "where does this person's data live?" instead of guessing.
- **Reliable change/delete/export:** Delta Lake's transactional guarantees (ACID transactions) mean that when a deletion job runs, it either fully succeeds or fully fails — you never end up in a half-deleted, inconsistent state.
- **Built-in housekeeping:** Z-ORDER (a way of physically co-locating related data on disk for faster filtering) and VACUUM (a command that permanently removes old, no-longer-needed data files) keep the system efficient and make sure deleted data doesn't linger.
- **Transaction log for auditing:** Delta Lake keeps a log of every change made to a table. That log itself becomes proof, during an audit, that a deletion request was actually carried out.

Term Check — Z-ORDER & VACUUM

Z-ORDER: a technique that sorts/co-locates similar data together in storage files so that filtering queries (e.g. "give me rows for country = India") skip reading irrelevant files, making queries faster. VACUUM: a Delta Lake command that permanently deletes old, unreferenced data files that are no longer needed for time-travel/history — this is the step that actually frees the storage and removes data for good.

1.3 The Three Pillars of Data Privacy

The course frames data privacy work as three sequential pillars. Think of it as a lifecycle: you can't protect what you haven't found, and you can't manage what you haven't protected.

Pillar 1 — Identify

Before you can protect anything, you need a clear map of your data:

- What data exists, where does it come from, and where does it end up being consumed?
- How is that data actually handled day to day?
- How confident are you that you've correctly detected everything that needs protecting?
- How do you know protection has actually been applied?

In practice, "Identify" is implemented through Data Discovery (comprehensive inventories of what sensitive data you hold), Data Classification (labelling data as PII, financial, health, etc.), and Data Mapping (documenting how data flows: where it's stored, who can access it, how it's used).

Pillar 2 — Protect

Once you know what you have, you decide how to guard it. Key questions: Should you do nothing (accept the risk)? Should you anonymize or pseudonymize? Can you still get the original data back if you need to?

This pillar is implemented through Data Security Measures (encryption, access controls, firewalls), Data Handling/Minimization (only collect what you truly need — a core GDPR principle), and — outside Databricks' scope but still worth knowing — Consent Management and Privacy by Design (building privacy in from the start of a project rather than bolting it on later).

Pillar 3 — Manage

Managing is the ongoing operational layer: how do you search across all your data, and how do you actually action a user's rights ("to be forgotten," "to be rectified," "to restrict processing")?

This is implemented through Data Governance (policies and procedures for the whole data lifecycle), Compliance Management (adherence to GDPR/CCPA/HIPAA), and Ongoing Monitoring & Auditing. Two more pieces — Data Subject Rights Management (formally handling access requests) and Incident Response (breach response plans) — sit outside what Databricks itself provides, but your organization still needs them.

Exam Tip

If a question asks you to classify an activity as Identify / Protect / Manage — remember the order matches real life: find it → guard it → operate on it long-term. "Tagging sensitive columns" = Identify. "Encrypting a column" = Protect. "Running a monthly compliance audit" = Manage.

Part 2 — Unity Catalog: Key Concepts & Components

2.1 Life Before Unity Catalog (The Problem)

To appreciate why Unity Catalog (UC) exists, it helps to see the mess it replaced. In a typical pre-UC enterprise, governance was scattered across every layer of the stack:

- **Data lakes (e.g. AWS S3):** Permissions are set on files and directories, not on rows/columns. That's coarse-grained — you cannot say "this group can see only rows for India." The only workaround is physically duplicating data into separate folders per policy, which becomes a maintenance nightmare the moment a rule changes (e.g., a new state passes its own privacy law).
- **Metadata layer (e.g. Hive Metastore):** Table/view permissions are managed separately from file permissions — and the two can silently drift out of sync. A user might have file access but not table access, or vice versa, with no guarantee they match.
- **Data warehouses:** Offer finer-grained control (table/column/view level) but run on a completely different governance model than the lake — so the moment data moves between lake and warehouse, you've created a data silo with two inconsistent rulebooks.
- **Other assets:** Dashboards, ML models, and notebooks each bring their own separate permission systems and tech stacks — and this gets worse again across multiple clouds.

The net effect: inconsistent, error-prone governance that's hard to audit and hard to reason about. This is exactly the gap Unity Catalog was built to close.

2.2 What is Unity Catalog?

Unity Catalog is Databricks' unified governance layer for all data and AI assets in the Lakehouse — tables, views, volumes (files), ML models, notebooks, dashboards, and more — all managed through one consistent permission model and one audit trail, regardless of which cloud you're on.

> **In Plain English**
>
> Before UC, every room in your house had a different lock and a different key system (one for the data lake, one for the warehouse, one for dashboards...). Unity Catalog installs a single master lock-and-key system for the whole house, so one security guard (one set of rules) can govern everything.

2.3 The Data Security Model: Access Control Lists (ACLs)

Unity Catalog controls access using ACLs, which are built from three components, easiest remembered as Who / What / How:

Component	Answers	Example
Principals ("Who")	A user, service principal, or — Databricks' recommended practice — a group	the group analysts
Securables ("What")	The object the grant applies to: catalog, schema, table, view, volume, function, model, etc.	the table sales.customers

Component	Answers	Example
Privileges ("How")	The specific permission type. Not every privilege applies to every object — e.g. MODIFY is meaningless on a view, since views are read-only.	SELECT, MODIFY, CREATE

Databricks strongly recommends granting privileges to groups rather than individual users — it's far easier to maintain (add/remove one person from a group) than to hand-manage dozens of individual grants.

Managing ACLs — Three Ways

The chapter shows three different interfaces for granting/revoking access, all backed by the same underlying model:

- **1. SQL (GRANT / REVOKE):** Run directly in a notebook or Databricks SQL.

```
GRANT SELECT ON TABLE t TO analysts;
REVOKE SELECT ON TABLE t FROM analysts;
```

- **2. Catalog Explorer (UI):** A point-and-click interface available in the workspace and in Databricks SQL — good for interactive, one-off changes.
- **3. Programmatically:** At the lowest level, everything can be managed via REST APIs. The Databricks CLI wraps those APIs into a more convenient command-line tool. And Terraform — an open-source Infrastructure-as-Code tool — lets you manage access (and most of the platform) using declarative configuration files.

> **Term Check — What is Terraform?**
> Terraform is an open-source tool (by HashiCorp) for "Infrastructure as Code": instead of manually clicking through a UI to create a catalog, a cluster, or a grant, you write a configuration file describing what you want to exist, and Terraform figures out how to create/update/delete real infrastructure to match that file. For Databricks, this means your catalogs, schemas, grants, clusters, and jobs can all be defined as version-controlled code — reviewed like any other code change, and repeatable across dev/staging/prod environments instead of manually recreated by hand each time. This matters for compliance too: a Terraform file is itself an auditable record of exactly what access was supposed to exist.

2.4 Automated Lineage for All Workloads

Unity Catalog automatically captures runtime data lineage — end-to-end visibility into how data actually flows and gets consumed — every time a query runs on a Databricks cluster or SQL warehouse. This spans tables, columns, dashboards, Lakeflow Jobs, notebooks, files, external sources, and even ML models.

This answers two questions that used to require a lot of manual detective work: "Where did this data come from?" and "What downstream things break if I change this pipeline?" Since lineage is captured automatically just by using UC, the best practice is simply: upgrade to Unity Catalog to get it for free.

2.5 Tagging & AI-Generated Documentation

Unity Catalog can auto-generate concise table/column comments (using AI) — useful for quickly documenting a backlog of undocumented tables — and lets you tag data assets (catalog, schema, table, or column level) to

make them easier to discover and to drive tag-based policies (e.g., "anything tagged pii must have masking applied").

2.6 Built-In Search & Discovery

A unified search UI lets users find data assets registered in Unity Catalog. Crucially, this search respects the same permission model as everything else — if you don't have access to see a table, it simply won't appear in your search results. The recommended best practice is to tag and comment your data on ingest, as part of normal pipeline operations, so the whole lakehouse stays discoverable over time.

2.7 Fine-Grained Access Control — Three Common Use Cases

"Fine-grained" means control that goes below the level of an entire table — down to specific rows or specific columns, for specific users or groups:

Use Case	What Happens	Typical Example
1. Limit access to columns	The column exists in the schema but its values are hidden/redacted for everyone except approved users/groups.	Hide a salary column from most analysts
2. Limit access to rows	Rows are entirely omitted from query results for certain users/groups based on filter criteria.	A regional manager only sees rows for their own region
3. Data masking	A special case of column restriction — instead of fully hiding the value, it's transformed/partially obscured.	Show only the last 4 digits of an SSN, or only the domain of an email

Masking is useful because it keeps just enough information to disambiguate results (e.g., "this is a Gmail user") without revealing the full protected value. Note that you could achieve similar effects by physically duplicating a table into a redacted copy — but that creates duplication, extra complexity, and ongoing maintenance headaches (two tables to keep in sync). UC's fine-grained control avoids this by protecting the original table directly.

2.8 Two Implementation Approaches

Databricks gives you two different mechanisms to achieve the same three use cases above. They behave the same from the end user's perspective but differ in how you build/maintain them.

Approach A — Dynamic Views

A dynamic view is a normal SQL view, but its definition includes conditional logic (typically using functions like is_member()) that changes what's returned depending on who's running the query. As long as the view's owner has access to the underlying table, people querying the view don't need direct access to that table — the view itself becomes the protective layer, filtering or transforming rows/columns on the fly. Anyone querying only ever sees what the view chooses to expose.

Approach B — Row Filters & Column Masks

This newer, more "native" approach attaches governance logic directly to the base table, rather than through a separate view:

- **Row Filtering:** A user-defined function (UDF) is attached to a table using SET ROW FILTER. The function runs for every query and decides which rows are allowed through — e.g. a function called us_filter using is_member() to check group membership.

```
-- conceptual example of a row filter UDF
CREATE FUNCTION us_filter(region STRING)
RETURN IF(is_member('us_team'), true, region = 'US');

ALTER TABLE sales SET ROW FILTER us_filter ON (region);
```

- **Column Masking:** A masking function is attached to a specific column using SET MASK. Every reference to that column is transparently replaced with the function's output — e.g. a function ssn_mask that uses a CASE expression to return either the real SSN (for authorized users) or a redacted version.

```
-- conceptual example of a column mask UDF
CREATE FUNCTION ssn_mask(ssn STRING)
RETURN CASE WHEN is_member('hr_admins') THEN ssn
            ELSE CONCAT('XXX-XX-', RIGHT(ssn, 4)) END;

ALTER TABLE employees ALTER COLUMN ssn SET MASK ssn_mask;
```

Dynamic Views vs. Row Filters/Column Masks — Which to Use?

	Dynamic Views	Row Filters & Column Masks
How it's applied	A new view object wraps the base table	Attached directly onto the base table's definition
Maintenance	You must create and maintain a separate view for every distinct access pattern you need	You create UDFs and attach them once — no duplicate view objects to track
End-user experience	Users query the view, not the table	Users can query the table directly and still be governed
End result	Same protective outcome (hide/filter/mask)	Same protective outcome (hide/filter/mask)

Exam Tip

If a scenario says "without creating extra view objects" or "apply masking directly to the table," the answer is Row Filters / Column Masks. If it says "create a separate object that exposes a restricted version

	Dynamic Views	Row Filters & Column Masks
	of the data," that's a Dynamic View.	

2.9 Data Governance in the Real World

The chapter frames governance as fundamentally a People–Process–Tools problem — Unity Catalog helps with the Process and Tools part, but people and organizational discipline still matter. The key operational questions it poses:

- What data assets do we have? → a Centralized Catalog answers this.
- Who can access the data? → Centralized ACLs answer this.
- Who has been accessing the data? → Auditing answers this (Part 3).
- How is data flowing across the estate? → Data Lineage answers this.
- How is data used, and where can we clean up? → Data Insights answer this.
- How is my data quality over time? → Lakehouse Monitoring answers this.

Worked example from the slides — a large healthcare provider: without proper governance, a misconfigured database could expose patient records (privacy breach + legal risk), outdated records could cause incorrect diagnoses, and poor data quality could block valuable research. With Unity Catalog-driven governance, access controls reduce breach risk, automated quality checks improve accuracy, and easier discovery accelerates research collaboration.

Part 3 — Auditing Your Data with System Tables

Unity Catalog ships with a built-in system catalog full of ready-made tables that let you query the operational state of your own lakehouse — who owns what, who accessed what, and what it all costs. This turns "governance questions" into simple SQL queries instead of guesswork.

3.1 Object Metadata — information_schema

Found under the information_schema schema, these tables let you run analytics on your own catalog structure. Typical questions you can answer:

- What tables exist in a specific catalog?
- Who owns a given table?
- Who has access to a specific table?
- Who last updated a gold table, and when?

> **Term Check — Information Schema**
> information_schema is a standard concept (not unique to Databricks) — a set of read-only, system-generated views that describe the metadata of a database itself (its tables, columns, privileges) rather than describing business data. Think of it as "the database describing itself."

3.2 Billing Logs

Found in the billing schema (specifically the usage table), these system tables let you understand cost allocation across your whole data estate:

- What's the daily trend in DBU consumption?
- How many DBUs of each SKU were used this month?
- Which 10 users consumed the most DBUs?
- Which jobs consumed the most DBUs?

> **Term Check — DBU**
> A DBU (Databricks Unit) is Databricks' unit of processing capacity used for billing — think of it like a "compute credit" that gets consumed per second based on the size/type of cluster you run. Different workload types (jobs, all-purpose clusters, SQL warehouses) consume DBUs at different rates.

3.3 Audit Logs (Public Preview)

Found in the access schema (audit table), these near-real-time logs let you see who accessed what, and when:

- Which table has the most access?
- Who deleted a specific table?
- What has a particular user accessed in the last 24 hours?
- What tables does a specific user access most frequently?

Because this feature was in Public Preview at the time of recording, always check with your workspace admin to confirm it's enabled before relying on it.

3.4 Lineage System Tables (Public Preview)

Also under the access schema, split into table_lineage and column_lineage tables, letting you query upstream/downstream relationships directly with SQL instead of only viewing lineage graphs in the UI:

- What tables are sourced from a specific table?
- What user queries read from a specific table?

> **Exam Tip**
> Remember the schema groupings: information_schema → structure/ownership questions. billing → cost questions. access → who-did-what (audit) and where-does-it-flow (lineage) questions. If an exam question gives you a specific question type, match it to the right system schema.

Part 4 — Data Isolation in Unity Catalog

This section covers how Unity Catalog physically and logically organizes your data assets, and how to design boundaries that keep the right data away from the right people/environments.

4.1 The Data Hierarchy

Unity Catalog organizes everything using a three-level namespace, with a Metastore sitting above all of it:

```
Metastore
   └── Catalog  ── (also: Share / Recipient / Provider / Connection)
         └── Schema
               └── Table / View / Volume / Model / Function
```

A fully qualified name always follows catalog.schema.object — for example sales.gold.customer_ltv. This is the same three-level pattern used across most modern data warehouses, so if you've used catalog.schema.table naming before, this will feel familiar.

4.2 Metastores

The Metastore is the top-level metadata container. It tracks tables, views, and volumes, and the permissions that govern access to them.

- **Regional scope:** Account admins create one metastore per cloud region, and it can be assigned to multiple workspaces within that region.
- **Regional isolation, not data isolation:** This is a subtle but important point — metastores give you regional separation (e.g., data physically stays in the EU), but they are NOT meant to be your unit of fine-grained data isolation. That job belongs one level down, at the catalog.

4.3 Catalogs — The Real Unit of Isolation

Catalogs are the primary unit of data isolation in Unity Catalog. In practice, teams often create catalogs that mirror organizational units (e.g., one per business unit) or software lifecycle stages (e.g., separate dev, staging, and prod catalogs).

- **Storage:** A catalog can share the metastore's default storage location, but Databricks prefers giving each catalog its own separate storage — this makes it much easier to fully isolate one catalog's data from another's, physically as well as logically.
- **Workspace binding:** A catalog can be bound to specific workspaces only, ensuring sensitive data can only ever be processed in an approved environment.
- **Inherited permissions:** Because permissions granted at the catalog level cascade down to schemas and tables inside it, the catalog is the natural place to set broad, inherited access rules.

4.4 Volumes — Governance for Non-Tabular Data

Volumes extend Unity Catalog's governance to files that are not structured rows and columns — think of libraries, config files, or Structured Streaming checkpoint folders.

- Volumes can hold structured, semi-structured, or fully unstructured data.
- They provide the same governance benefits (access control, discoverability) that tables get, but for files.
- Important distinction: data sitting in a volume cannot be registered or queried as a table — it stays file-shaped.
- Volumes can be managed (Databricks controls the storage location) or external (pointing at a location you already manage in your own cloud account).

4.5 Physically Separating Data & Governance Models

Organizations sometimes have hard requirements that certain data types must live in specific cloud accounts or storage buckets. Unity Catalog supports this by letting you configure distinct storage locations at the metastore, catalog, or schema level.

Beyond storage location, UC also supports two governance operating models:

Model	How it Works	Best For
Centralized	A governance admin team owns the whole metastore, can take ownership of any object, and grants/revokes all permissions centrally.	Organizations wanting one team fully in control of all policy
Distributed (Data Domains)	Each catalog (or set of catalogs) is treated as an independent "data domain." The owner of that catalog manages governance within it, independently of other domains.	Larger orgs where different business units self-manage their own data

Regardless of which model you choose, Databricks strongly recommends assigning a group (not an individual person) as the metastore admin or catalog owner — so governance doesn't depend on one person's account.

4.6 External Locations & Storage Credentials

An external location is simply a path to your cloud storage combined with a storage credential (the cloud-level permission that actually allows reading/writing there). Together, they let Unity Catalog read and write data in your own cloud tenant, and are used specifically to register external tables and external volumes (as opposed to UC-managed ones).

Because external locations are powerful, best practice is to restrict direct user access to the underlying cloud storage container itself — otherwise a user could simply bypass Unity Catalog entirely and go straight to the cloud storage, defeating every ACL you've set up.

4.7 How a Query Actually Flows Through Unity Catalog

It's worth walking through the full lifecycle of a single query, because this shows exactly where security is enforced:

- **1. A principal issues a query.** This could be a user on an all-purpose cluster (interactive Python/SQL), a job/pipeline running as a service principal on a job cluster, or an analyst/BI tool querying through a SQL warehouse.
- **2. The request is sent to Unity Catalog.** UC logs the request and validates it against every security rule defined in the relevant metastore.
- **3. UC assumes the right cloud credential per object.** For each object referenced in the query, UC picks up the appropriate cloud credential (managed storage for managed tables, or the relevant storage credential for external tables/files).
- **4. UC issues a short-lived, scoped token.** For each referenced object, UC generates a temporary token plus an access URL — this lets the compute layer read the data directly from cloud storage, without ever handing out a long-lived, broad credential.
- **5. Compute fetches data directly from storage.** The cluster or SQL warehouse uses that URL+token to pull data straight from cloud storage, repeating this for every object in the query.
- **6. Last-mile filtering happens on compute.** Once data reaches the cluster/warehouse (already narrowed to the relevant partitions), row/column filtering (masks, row filters) is applied as the very last step.
- **7. The filtered result goes back to the caller.**

> **Why This Matters**
> This design means Unity Catalog never actually moves your data through itself — it's a governance/authorization layer that hands out short-lived permission slips, while the real data transfer happens directly between cloud storage and compute. That's how it stays secure without becoming a performance bottleneck.

4.8 Data & Workload Isolation via Clusters and Workspaces

Beyond UC's own controls, you can isolate data using compute and workspace boundaries:

- **Single-user clusters:** Provide the strongest data protection — only one person's identity is ever active on that cluster.
- **Multi-user / shared-access clusters and SQL Warehouses:** Support many users with different privilege levels safely at once, because Unity Catalog enforces per-user permissions even on shared compute.
- **Multiple workspaces:** A good way to isolate groups of people who should never collaborate or share data at all — e.g., separate workspaces per business unit or per client.

A typical setup: Workspace 1 might have one single-user cluster (for a proof-of-concept) plus one shared-access cluster (for an active team project), while Workspace 2 runs its own SQL Warehouse for cross-team collaboration.

4.9 Encryption by Default

Databricks encrypts data by default across the whole platform, without any special configuration required:

- All traffic between the control plane and the data plane (compute plane).
- All traffic between the user and the control plane.

- All storage in the Databricks account.
- All attached storage volumes (e.g., EBS volumes on AWS).
- All traffic to cloud provider APIs (e.g., AWS APIs).

Encryption Type	What It Covers	How
Data in transit	Traffic between users ↔ control plane ↔ compute plane ↔ cloud APIs	TLS/SSL protocols
Data at rest	Data stored on cloud storage / EBS volumes	AES-256 encryption
Control plane data	Metadata Databricks itself manages	Envelope encryption — a Data Encryption Key (DEK) is encrypted with a Customer-Managed Key (CMK), which is itself re-encrypted with a Databricks-managed key

Term Check — Envelope Encryption / CMK / DEK

Envelope encryption is a layered approach: instead of encrypting your data directly with your master key (risky — that key gets used constantly), you generate a one-time Data Encryption Key (DEK) to encrypt the actual data, and then encrypt that DEK using your more protected Customer-Managed Key (CMK). It's like putting your valuables in a small lockbox (DEK-encrypted data), then locking that lockbox inside a bigger, more heavily guarded vault (CMK). Databricks adds

Encryption Type	What It Covers	How
	one more layer by also wrapping things with its own managed key.	

4.10 Managing Access to PII — Practical Checklist

Beyond ACLs, the chapter recommends several complementary layers of defense:

- Control access to storage locations directly with cloud-native permissions (don't rely solely on Databricks-level controls).
- Limit human access to raw data wherever possible.
- Use Unity Catalog row and column filtering.
- Pseudonymize records as early as possible — ideally right at ingestion.
- Use table ACLs to manage who can do what.
- Configure dynamic views for redaction where appropriate.
- Remove identifying details from demographic-style views.

An important operational detail: because Databricks clusters and their attached storage are ephemeral (they get torn down after a job finishes), cached data doesn't linger by itself. But if you're persisting PII to durable storage, removing natural keys that could link back to a real identity limits how many places you'd ever need to go delete data from later.

4.11 New Best Practices (Post-Unity-Catalog)

The chapter lists six modernized best practices now that Unity Catalog exists — essentially: stop doing things the "old," pre-UC way.

- **1. Avoid direct access to object stores.** Let UC mediate all data access so every request lands in one audit trail, instead of users going straight to cloud storage.
- **2. Minimize keys in circulation.** Fewer credentials floating around means fewer opportunities for a leak — UC's centralized model reduces how many separate access keys you need at all.
- **3. Eliminate credentials in code or secret scopes.** Store sensitive values using Databricks Secrets rather than hardcoding them or scattering them across notebooks.
- **4. Migrate off the Hive Metastore.** Hive Metastore is considered less secure than Unity Catalog — new data assets should be created directly in UC.
- **5. Move ML models into UC.** Store model assets as schemas inside Unity Catalog rather than in workspace-local model registries, for consistent governance and cross-workspace sharing.
- **6. Avoid DBFS (Databricks File System).** In UC-enabled workspaces, use Volumes for unstructured data (checkpoints, libraries, configs) instead of the legacy DBFS.

4.12 Real-World Case Study — SEEK

SEEK, Australia's largest online employment marketplace, processes millions of resumes containing sensitive candidate information. Their goal was a near-real-time surveillance system that spots governance gaps, builds dashboards, and automatically opens tickets for data custodians to fix issues.

- They built a custom PII-detection framework using HuggingFace transformer models, working across both structured and unstructured data.
- Their detection pipeline includes custom recognizers for specific business contexts, OCR to catch PII inside uploaded documents (like scanned passports), MLflow integration for model versioning, and real-time alerts that warn users before they submit sensitive information.
- Unity Catalog underpins the whole system for lineage tracking and enforcing access control.

The bigger takeaway: this shifts data governance from a manual, reactive process into an AI-driven, automated one — improving both privacy protection and risk management at scale.

Part 5 — PII Data Security: Pseudonymization & Anonymization

A very important caveat opens this section: with enough time and enough auxiliary data, almost any dataset can eventually be re-identified, no matter which technique you use. Pseudonymization and anonymization reduce risk and reduce exposure — they don't provide a 100% guarantee. Keep this in mind for exam questions that test whether a technique is "reversible" or "fully safe."

5.1 The Two Approaches at a Glance

	Pseudonymization	Anonymization
Scope	Protects data at the record level	Protects an entire dataset (table, database, or catalog)
Reversible?	Yes — re-identification is possible by an authorized party	No — irreversibly altered; not linkable back to the original
Still "personal data" under GDPR?	Yes — pseudonymized data is still legally considered PII	No — properly anonymized data generally falls outside PII scope
Typical use	Machine learning on realistic-looking but de-identified records	Business Intelligence / aggregate reporting where individual reversal isn't needed

Worked mini-example from the slides: the name "John Doe" becomes "User-321" under pseudonymization — a reversible key that could be joined back to the original later — versus a fully anonymized version where the record can no longer be traced to a specific person at all.

5.2 Pseudonymization Deep Dive

Pseudonymization replaces PII with artificial identifiers (pseudonyms) that authorized users can reverse when truly necessary. It protects data at the record level, which is exactly what makes it useful for machine learning: a data scientist can work with complete, realistic-looking records without ever seeing the literal underlying values.

There are two main methods:

Method A — Hashing

A hash function (like SHA-256) turns any input into a fixed-length, seemingly-random string. Because the same input always produces the same hash, this is deterministic — which is both useful (you can still JOIN on the hashed value) and risky (an attacker with a big dictionary of possible values could reverse it by trial and error).

- **Salting:** To defend against that risk, you append a random string (a "salt") to the value before hashing it. This means the same SSN hashed with two different salts produces two completely different outputs, making dictionary/rainbow-table attacks much harder.
- **Where to store the salt:** Databricks Secrets can store the salt value securely, restricting read access to only production jobs and authorized users, and ensuring the salt is never displayed in plain text inside a notebook.

- **Trade-offs:** Hashed values take up more bytes than the original data (increasing storage size), and some downstream operations become less efficient. If a later ML step needs a derived piece of information (e.g., just the domain of an email address), extract that piece before hashing and store it in its own separate (and separately hashed) column — otherwise you lose the ability to use it.

Method B — Tokenization

Tokenization converts every PII value into a token (a key), and stores the real value separately in a secure lookup/vault table. The table that end users actually query only contains these tokens (usually simple long integers) — never the real values.

- **Trade-off profile:** Tokenization is slow to write (every new unique value must be registered in the vault) but fast to read, and results in a smaller de-identified dataset since a token is typically much smaller than the original value it replaces.
- **How it works mechanically:** All relevant columns get turned into an array of structs; each unique value across that array gets assigned a token; the vault stores value↔token mappings; and the public-facing table only ever exposes the token.

	Hashing	Tokenization
Speed	Fast to write and read (just compute a hash)	Slow to write (vault lookup/insert), fast to read
Storage size	Increases (hash values are larger than most raw values)	Decreases (tokens are typically compact)
Reversal	Only reversible by brute-force / dictionary attack unless you keep a lookup — one-way by design	Reversible by design, via the secure vault table
Best for	Values you need to match/join on, but never need to "look up" back to original	Values where authorized re-identification is a real, expected workflow

5.3 Anonymization Deep Dive

Anonymization irreversibly alters personal data so that a data subject can no longer be identified — directly or indirectly. It protects entire datasets and is mostly aimed at Business Intelligence use cases, since analysts usually care about aggregate trends, not individual records. Real-world anonymization typically layers multiple techniques together rather than relying on just one.

Method A — Data Suppression

Suppression means simply removing the risky pieces entirely: excluding PII columns from views altogether, or removing rows where a demographic group is too small to hide within (a group of one is trivially re-identifiable). Dynamic access controls can grant conditional access to the fuller data only for approved roles.

> **In Plain English**
> If a company has exactly one customer in a small town, showing "average purchase in [that town]" is really just showing that one person's purchase. Suppression means dropping rows like that from reports rather than exposing them.

Method B — Generalization

Generalization reduces the specificity/precision of data rather than removing it outright. The chapter covers four flavors:

- **Categorical generalization:** Move from a specific category to a broader one — e.g., "Rock music" → "Music", or a specific city like "Haarlem" → the country "The Netherlands". This preserves useful signal while hiding the exact detail that could re-identify someone (e.g., you being the only survey respondent from your specific team).
- **Binning:** Group continuous values into meaningful ranges — e.g., turning individual ages into 10-year age bands, or exact salaries into salary bands based on published pay standards. Domain expertise helps decide where the bin boundaries should fall so the groups stay analytically useful.
- **Truncating IP addresses:** Zero out the last byte of an IP address (rounding it into a /24 CIDR block). This still tells you the rough city/neighborhood-level location without pinpointing the exact device.
- **Rounding:** Apply a consistent rounding rule to numeric data — e.g., round every value to the nearest 5. General trends survive because values get rounded up and down roughly equally, but you should still watch for outliers at the extreme high/low end, which may need to be suppressed separately even after rounding.

5.4 Best Practices for Handling PII

The chapter closes this topic with ten practical rules of thumb — genuinely worth memorizing for the exam:

- **1–3. Prefer stronger protection:** Having no PII at all beats having PII. When you must have PII: Anonymization > Pseudonymization > storing it in clear text — always in that order of preference.
- **4. Stay paranoid:** Maintain a healthy skepticism about whether the protections you've applied are actually sufficient.
- **5. Apply the "three facts" rule:** Be alert that combining just three pieces of "harmless" information about someone is often enough to re-identify them (e.g., ZIP code + birth date + gender has famously been shown to uniquely identify most U.S. individuals).
- **6. Think about dataset combination risk:** Consider how your "anonymized" dataset could be re-identified if joined against some other, seemingly unrelated dataset (this is exactly how the famous Netflix Prize dataset was de-anonymized by cross-referencing it with public IMDb reviews).
- **7. Train your teams:** Make sure data teams actually understand the privacy laws that apply to them, not just the tools.
- **8. Not all PII is equal:** A person's favorite color is far less sensitive than their SSN or health records — calibrate protection effort to actual sensitivity.
- **9. Conduct PIIR reviews:** Run regular Privacy Impact Assessment Reviews to proactively catch and mitigate risk.
- **10. Isolate PII-processing environments:** Keep the systems that touch PII separate from everything else — this both adds a security layer and makes compliance audits dramatically simpler, since you know exactly where to look.

Exam Tip

> Memorize the ordering: Anonymization > Pseudonymization > Cleartext. If a scenario asks "which is the most protective option for this dataset," and full anonymization is feasible, that's almost always the preferred exam answer over pseudonymization.

Part 6 — Streaming Data & Change Data Feed (CDF)

6.1 The Core Problem: Streaming Expects Append-Only

Structured Streaming treats a data source as a table that only ever grows — it's designed around the assumption that data is append-only. But real life doesn't cooperate: source systems issue updates and deletes all the time, and this breaks that core assumption.

Adding to the challenge, Delta Lake's transaction log tracks changes at the file level, not the row level. So updating a single row doesn't create a tidy "one row changed" record — it actually produces an entirely new version of whatever file that row lived in. Without extra logic, a naive stream reader has no clean way to tell "this is a brand-new row" apart from "this file changed because one existing row inside it got updated."

This matters directly for PII/compliance work: Structured Streaming is otherwise a great way to incrementally propagate updates and deletes across a whole chain of tables in a pipeline — but only once you solve the update/delete problem.

6.2 Solution 1 — Ignore the Changes

The simplest fix: explicitly tell the stream to skip over anything that isn't a pure insert. This preserves the append-only assumption at the cost of losing visibility into updates/deletes (which you'd then need a separate mechanism to handle, if you need them at all).

Option	What It Does	Syntax
ignoreDeletes	Ignores transactions that delete data at partition boundaries only; no new files are written for full-partition removals	spark.readStream.format("delta").option("ignoreDeletes", "true")
skipChangeCommits	Discards ALL file-changing operations (updates and deletes both) and returns only inserted rows. This supersedes/subsumes ignoreDeletes.	spark.readStream.format("delta").option("skipChangeCommits", "true")

Term Check

You may also see the older option name ignoreChanges in documentation — it's being

Option	What It Does	Syntax
	deprecated in favor of skipChangeCommits, which does the same job with clearer intent. If you see "ignoreChanges" on the exam, mentally treat it as the predecessor to skipChangeCommits.	

This approach fits when your use case genuinely only cares about new data arriving, and you're comfortable handling any necessary update/delete logic through a completely separate process.

6.3 Solution 2 — Change Data Feed (CDF)

The more powerful option: Change Data Feed (CDF) tracks row-level changes between versions of a Delta table — including both the actual row data and useful metadata about the change. Unlike the "ignore" options, CDF doesn't throw information away; it captures it in a structured, queryable form.

- **Enabling it:** CDF must be manually turned on, either at table creation time or afterward using ALTER TABLE (see 6.8 below for exact syntax).
- **Typical flow:** Bring external sources into the Bronze layer, enable CDF from that point on, and then use the feed to move changes efficiently into Silver/Gold, or even feed them out to an external platform.
- **Cost:** CDF isn't free — it adds some storage/processing overhead for the extra change-tracking metadata it needs to keep.

6.4 What CDF Is Actually Used For

Use Case	How CDF Helps
Improve ETL pipelines	Processes only the rows that actually changed, instead of reprocessing entire tables — increasing pipeline efficiency

Use Case	How CDF Helps
Unify batch & streaming BI	Provides one common change format that works whether your downstream consumer reads in batch or streaming mode, supporting inserts, updates, and deletes consistently
Meet regulatory needs	Gives a full, queryable history of everything that changed in the data — including deletions — supporting audit and "right to be forgotten" style requirements

A few more specific applications worth knowing:

- **Silver & Gold tables:** Propagating CDF output into these layers keeps them current with substantially less processing cost than a full re-read.
- **Materialized Views:** When you need an aggregated view of gold-level data for a dashboard, CDF lets you avoid full, expensive re-aggregations while still reflecting changes correctly.
- **Transmitting changes to other systems:** For downstream platforms that can ingest a change feed, this becomes a low-overhead way to incrementally keep other databases/applications in sync.
- **Audit trail tables:** Storing CDF output in its own Delta table gives compliance/audit teams a quick, queryable answer to exactly what happened to a record, a set of records, or an entire table.

6.5 CDF vs. CDC — Don't Confuse These

CDC (Change Data Capture) is the general industry concept of capturing changes made to data at the source. CDF is Databricks/Delta Lake's specific, native implementation of that concept, for Delta tables.

Feature	CDF (Change Data Feed)	CDC (Change Data Capture — general concept)
Scope	Specific to Delta Lake tables	A general concept, applicable across many different database/storage systems
Implementation	A native Delta Lake feature — just a config flag away	Typically needs external tools or custom logic; in Lakeflow Spark Declarative Pipelines this is implemented via the APPLY CHANGES INTO syntax
Efficiency	Processes only the rows that changed	Depends entirely on the specific tool/implementation used

Exam Tip

If you see APPLY CHANGES INTO in a question, that's the Lakeflow Spark Declarative Pipelines syntax that implements CDC-style processing (handling out-of-order updates/deletes

Feature	CDF (Change Data Feed)	CDC (Change Data Capture — general concept)
	automatically). CDF (table_changes(), the _change_data folder) is the lower-level Delta Lake primitive you could use to build similar behavior yourself in plain Structured Streaming.	

6.6 How Delta CDF Actually Works Internally

Walk through a concrete example: start with an original table with three rows (A1, A2, A3) and two columns (PK, B). A new version arrives that changes field B in A2, deletes A3, and adds a new row A4.

- CDF captures only the rows that actually changed — A2 (updated), A3 (deleted), A4 (inserted). Row A1 does not appear at all in the CDF output, because nothing about it changed.
- For updates specifically, CDF emits two rows: the preimage (what the record looked like before the change) and the postimage (what it looks like after). This lets downstream consumers apply precise, surgical updates to materialized views or aggregated facts without reprocessing everything underneath them.
- For inserts and deletes, a single record is emitted, tagged with whichever change type applies.
- Every change record is stamped with its Delta version and a timestamp, which is what makes CDF genuinely useful for audits — you get exact granularity on when a change happened, not just that it happened.
- This mechanism works identically whether the source of the update was a batch job or a live stream.

CDF exposes these extra metadata columns on top of your normal row data:

Column	Meaning
_change_type	insert, delete, update_preimage (value before change), or update_postimage (value after change)
_commit_version	The Delta table version number this change belongs to
_commit_timestamp	The wall-clock time the commit happened

6.7 Consuming CDF: Two Patterns

Pattern A — Stream-Based Consumption

You use Structured Streaming to process the change feed continuously. Each micro-batch simply picks up whatever has landed since the last checkpoint — there's no fixed schedule to wait for. Worked example from the timeline in the slides:

- A large upsert lands at 12:00 → the stream immediately starts processing it.
- An insert (Delta version 3) lands at 12:08 → picked up right away in the next micro-batch, no waiting.
- Another insert (version 4) lands at 12:09, while the prior micro-batch is still running.
- An update to that same version-4 record (version 5) lands at 12:10:05. As soon as the previous micro-batch finishes, the next one starts and scoops up both version 4 and version 5 together.
- Because both an insert and a later update can land in the same micro-batch, your processing logic needs explicit handling to make sure you keep the latest state, not an outdated intermediate one.

Pattern B — Batch-Based Consumption

Here, you run the change-feed read on a fixed schedule (e.g., every 10 minutes) instead of continuously. You must track a high-watermark yourself — the latest Delta version or timestamp you've already processed — and only pick up changes after that point each time you run.

- A batch job scheduled every 10 minutes kicks off at 12:00 and processes the large upsert.
- It pauses, then resumes at 12:10 — checks the high-watermark (version 2), and this time picks up everything since then: both version 3 and version 4 together, in one pass.
- It pauses again and doesn't pick up version 5 until the 12:20 run — even though version 5 actually arrived shortly after the 12:10 run finished. This delay is simply the nature of fixed-interval batch processing.

> **Stream vs. Batch — Which to Choose?**
> Streaming reacts as soon as the prior micro-batch finishes — lower latency, but potentially more complex handling of "multiple changes to the same record in one batch." Batch is simpler to reason about and schedule but always carries some built-in latency (you're waiting for the next scheduled run, even if data is ready sooner).

6.8 Enabling & Configuring CDF

CDF is not enabled by default — you have to turn it on explicitly, either for one table or as a new default for every future table:

```
-- Enable CDF on an existing table
ALTER TABLE my_catalog.my_schema.my_table
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

-- Make CDF the default for every NEW table created in this session
SET spark.databricks.delta.properties.defaults.enableChangeDataFeed = true;
```

Once enabled, you can read the change feed by version range or by timestamp range — whichever is more convenient for your use case.

6.9 The table_changes() Function

This is the batch-mode entry point for reading CDF. It returns a log of changes (inserts, updates, deletes) for a Delta table between a start and optional end boundary, including the three metadata columns described above (_change_type, _commit_version, _commit_timestamp).

```
-- Syntax
table_changes(table_str, start [, end])

-- Example: read all changes between version 5 and version 10
SELECT * FROM table_changes('my_catalog.my_schema.my_table', 5, 10);

-- Example: read changes using timestamps instead of version numbers
SELECT * FROM table_changes(
  'my_catalog.my_schema.my_table',
  '2026-01-01T00:00:00',
  '2026-01-02T00:00:00'
);
```

For the streaming counterpart, Structured Streaming reads directly from the table's underlying _change_data folder (created automatically once CDF is enabled) rather than calling table_changes() explicitly.

Part 7 — Deleting Data in Databricks (The "Right to Be Forgotten" in Practice)

This section takes everything from Parts 1–6 and applies it to the single hardest privacy operation: actually deleting someone's data on request.

7.1 Why Deletion Needs Special Attention

Companies must handle deletion requests carefully to stay compliant with GDPR/CCPA, and PII deletion for a given user needs to be handled effectively and efficiently across the whole lakehouse — not just in one table. In practice, these deletion operations are usually run as their own dedicated pipeline, kept separate from normal ETL pipelines, so that a compliance-driven delete never gets tangled up with regular business logic.

This is exactly where CDF becomes the connective tissue: CDF data can be used to propagate a deletion action from where it happens (often a Bronze/Silver table) down into every downstream table that depends on it — conceptually similar to how you might filter a shared "multiplex" bronze table to drive several different downstream pipelines. Additions (new/updated user records) simply flow through as normal inserts into Silver; deletion events get routed to a separate pipeline built specifically to satisfy privacy requirements.

7.2 Commit Messages for Auditability

Delta Lake supports arbitrary commit messages, recorded straight into the Delta transaction log and visible later in the table's history — a simple but powerful auditing tool.

- Commit messages can be set at a global level, or specified individually as part of a specific write operation.
- Example use: label a data insertion by its processing type — e.g. "manual" vs. "automated" — so a later audit can immediately tell how a given row got there.

7.3 Propagating Data Deletion with CDF

Deletion requests can be automated using Structured Streaming triggers, and CDF can be used to separately identify exactly which records need to be deleted or modified in every downstream table.

> **Important Gotcha**
> Even after Delta Lake's history and CDF track a deletion, the deleted PII values are still physically present in older versions of the data (that's the whole point of time-travel/history — it keeps old snapshots around). Turning on CDF or deleting a row does not, by itself, make the old bytes disappear from disk.

The fix: you must actually run VACUUM to physically remove the old data files, and deleting at a partition boundary makes this process considerably more efficient (since Delta can drop whole files/partitions rather than rewriting partially-affected ones).

7.4 CDF Retention Policy & VACUUM

A few critical facts about how CDF and VACUUM interact:

- File deletion does not actually occur until you run VACUUM on the table — deleting a row is a logical operation in the transaction log first.
- CDF records follow the same retention policy as the table itself — so running VACUUM also deletes the associated CDF change records, not just the underlying data files.
- By default, Delta's engine protects you from yourself: it refuses to VACUUM anything with less than 7 days of retention, precisely so you don't accidentally destroy data you might still need for time-travel or recovery.

To satisfy a genuine, immediate deletion requirement (e.g., a legally-required erasure), you need to deliberately override that default safety net:

```
-- 1. Disable Spark's retention duration safety check
SET spark.databricks.delta.retentionDurationCheck.enabled = false;

-- 2. ALWAYS preview first with a dry run before deleting anything for real
VACUUM my_catalog.my_schema.my_table RETAIN 0 HOURS DRY RUN;

-- 3. Run the real VACUUM once you've confirmed the dry-run output
VACUUM my_catalog.my_schema.my_table RETAIN 0 HOURS;
```

> **Exam Tip**
> These VACUUM steps apply to plain Delta tables managed outside of Lakeflow Spark Declarative Pipelines. If your compliance flow runs inside a Declarative Pipeline, retention/cleanup is handled through the pipeline's own configuration rather than manual VACUUM commands.

7.5 Can You Run DML on a Streaming Table? (The GDPR Question)

This is the natural question that falls out of everything above: if streaming tables are supposed to be append-only, how do you ever satisfy a GDPR deletion request against one?

A Worked GDPR Pipeline Example

The slides walk through a concrete architecture: data is ingested from Kafka (which itself only retains messages for a limited time) as JSON files, streamed into a Bronze table using append-only semantics (appropriate, since this is a streaming table). From there, APPLY CHANGES INTO processes the data into a Silver table, correctly folding in updates/deletes.

- In Lakeflow Spark Declarative Pipelines, retention itself is configurable — you can set the pipeline's reset permission so the table doesn't get wiped and recomputed from scratch on every refresh. This matters specifically when you need a permanent deletion to actually stay deleted rather than being silently "un-deleted" by a future full refresh.
- Downstream, Silver and Gold layers can use Materialized Views with full refreshes to recompute values reflecting the deletion — though be aware this can get expensive for very large tables.

The Answer: DML Works Directly on Streaming Tables

Despite the append-only expectation of raw Structured Streaming, Databricks streaming tables do support INSERT, UPDATE, DELETE, and MERGE INTO directly. This is the mechanism that actually resolves the GDPR question above.

```
-- Ensure compliance with a data retention window
DELETE FROM my_live_tables.users
WHERE updated < current_time() - INTERVAL 3 YEARS;

-- Scrub PII from data already in the lake
UPDATE my_live_tables.users
SET email = hash(email, salt)
WHERE <condition>;

-- Append new data as usual
INSERT INTO my_live_tables.users
VALUES (3, hash(email, salt), current_time());
```

In other words: you can delete rows to satisfy retention windows, update/scrub columns to remove PII in place (e.g., re-hashing an email with a fresh salt), or simply keep appending new data — all directly against a streaming table.

The Exception: Materialized Views Do NOT Support DML

Materialized Views work fundamentally differently — they store a precomputed snapshot of query results as of the last refresh, purely for fast retrieval. Because of that nature, you cannot run direct INSERT, UPDATE, DELETE, or MERGE INTO against a materialized view.

```
-- This works: query results are precomputed for speed
SELECT COUNT(*) FROM sales_mv GROUP BY region;

-- This does NOT work: MVs have no direct DML path
-- DELETE FROM sales_mv WHERE ...   ✗
```

Instead, materialized views only ever update via a refresh mechanism — either automatically as part of a Lakeflow Spark Declarative Pipelines run, or manually (if the MV was created manually) using a REFRESH command.

	Streaming Table	Materialized View
Supports direct DML?	Yes — INSERT / UPDATE / DELETE / MERGE INTO all work	No — read-only from the outside
How it changes	Directly, via DML statements	Only via a refresh (automatic in pipelines, or manual REFRESH)
Typical role	Bronze/Silver ingestion & mutation layer	Precomputed, fast-read Gold-layer aggregation
GDPR deletion approach	Run DELETE/UPDATE directly against the table	Trigger (or wait for) a refresh so the MV recomputes without the deleted data

Exam Tip

	Streaming Table	Materialized View
	If a question describes needing to delete/scrub a specific user's data and the object in question is a Materialized View, the correct action is to trigger/wait for a refresh — not to attempt a DELETE statement directly against it. If the object is a streaming table, DELETE/UPDATE/ MERGE INTO are all valid direct options.	

Part 8 — Glossary of Key Terms (A–Z)

Every important term used across this chapter, defined in plain English. Use this section for quick lookups while revising.

- **ACL (Access Control List) —** Databricks' access model built from three parts — Principals (who), Securables (what), Privileges (how) — that together define who can do what to which object.
- **Anonymization —** Irreversibly altering an entire dataset so a person can no longer be identified, directly or indirectly. Not legally considered PII once done properly.
- **APPLY CHANGES INTO —** Lakeflow Spark Declarative Pipelines syntax that automatically applies CDC-style updates/deletes/inserts into a target table — the practical replacement for hand-rolled deduplication logic.
- **Binning —** A generalization technique that groups continuous values (age, salary) into ranges/bands to hide the exact value while preserving trend-level insight.
- **Catalog —** The primary unit of data isolation in Unity Catalog's three-level namespace (catalog.schema.object); often mirrors a business unit or environment like dev/prod.
- **CCPA (California Consumer Privacy Act) —** California's privacy law. Requires acknowledging a data request within 10 business days and resolving it within 45 days; violations can cost $2,500–$7,500 each.
- **CDC (Change Data Capture) —** The general industry concept of capturing changes made to data at their source. In Databricks, implemented via APPLY CHANGES INTO.
- **CDF (Change Data Feed) —** Delta Lake's native feature for tracking row-level changes (inserts/updates/deletes) between table versions, exposed via table_changes() or the _change_data folder.
- **CMK (Customer-Managed Key) —** An encryption key that you (the customer) control, used in envelope encryption to protect the underlying Data Encryption Key (DEK).
- **Column Mask —** A UDF attached to a table column (via SET MASK) that transparently transforms/redacts the value shown to unauthorized users.
- **Databricks CLI —** A command-line tool that wraps the Databricks REST APIs, giving a more convenient way to manage objects (grants, clusters, jobs, etc.) programmatically.
- **DBFS (Databricks File System) —** A legacy filesystem abstraction on Databricks; now discouraged in Unity Catalog-enabled workspaces in favor of Volumes.
- **DBU (Databricks Unit) —** Databricks' unit of processing capacity/billing — a "compute credit" consumed per second, at a rate depending on cluster type/size.
- **DEK (Data Encryption Key) —** A key used to directly encrypt data; itself gets encrypted by a CMK as part of envelope encryption.
- **Dynamic View —** A SQL view whose definition contains conditional logic (e.g. is_member()) so different users see different filtered/masked results from the same underlying table.
- **Envelope Encryption —** A layered encryption approach: data is encrypted with a DEK, and the DEK itself is encrypted with a CMK (and again with a Databricks-managed key).

- **External Location —** A cloud storage path combined with a storage credential, letting Unity Catalog read/write data in your own cloud tenant; used to register external tables/volumes.
- **GDPR (General Data Protection Regulation) —** The EU's data protection law. Requires responding to data subject requests within 30 days; fines up to 4% of global annual revenue or €20M, whichever is greater.
- **Generalization —** An anonymization method that reduces data precision (categorical generalization, binning, IP truncation, rounding) rather than removing data outright.
- **Hashing —** A pseudonymization method that runs PII through a deterministic hash function (e.g. SHA-256), typically combined with salting, to produce a fixed obscured value.
- **High-Watermark —** In batch CDF consumption, the latest Delta version/timestamp already processed — used to know where to resume on the next run.
- **Hive Metastore —** The legacy metadata store predating Unity Catalog; considered less secure and being phased out in favor of UC for new data assets.
- **ignoreDeletes —** A Structured Streaming read option that ignores delete transactions occurring at partition boundaries only.
- **Information Schema —** A standard set of read-only system views (information_schema) describing a database's own metadata — tables, columns, privileges — rather than business data.
- **Lineage (Data Lineage) —** Automatically captured, end-to-end visibility into how data flows — from source through transformations to every downstream consumer (tables, dashboards, jobs, models).
- **Materialized View (MV) —** A table-like object storing precomputed query results as of the last refresh; supports fast reads but no direct DML — only updates via refresh.
- **Metastore —** The top-level Unity Catalog metadata container, created one-per-region, providing regional isolation (not the main tool for data isolation — that's the catalog).
- **PII (Personally Identifiable Information) —** Any data that can identify a specific individual — names, SSNs, emails, etc. — subject to regulations like GDPR/CCPA.
- **PIIR (Privacy Impact Assessment Review) —** A regular, structured review process to evaluate and mitigate privacy risks in how data is collected, stored, and used.
- **Pseudonymization —** Replacing PII with artificial identifiers that authorized parties can reverse later; still legally considered personal data under GDPR.
- **REST API —** The lowest-level programmatic interface for managing Databricks objects, underlying both the CLI and Terraform provider.
- **Row Filter —** A UDF attached to a table (via SET ROW FILTER) that determines, per query, which rows are allowed through to the caller.
- **Salting —** Appending a random string to a value before hashing it, to defend against dictionary/rainbow-table reversal attacks on deterministic hashes.
- **Schema —** The level of Unity Catalog's namespace sitting between catalog and table (catalog.schema.table); groups related tables/views/volumes.

- **skipChangeCommits —** A Structured Streaming read option that discards all file-changing operations (updates AND deletes), returning only inserted rows; supersedes ignoreDeletes.
- **Streaming Table —** A table designed for continuous ingestion; despite the append-only expectation of raw streaming, Databricks streaming tables do support direct DML (INSERT/UPDATE/DELETE/MERGE INTO).
- **Structured Streaming —** Spark's engine for processing data streams as if they were continuously-appended tables; expects append-only sources by default.
- **System Tables —** Built-in tables under the system catalog (information_schema, billing, access schemas) that let you query metadata, cost, audit logs, and lineage using plain SQL.
- **table_changes() —** The SQL function used to read CDF output in batch mode for a version or timestamp range: table_changes(table_str, start [, end]).
- **Terraform —** An open-source Infrastructure-as-Code tool (by HashiCorp) that manages real infrastructure (including Databricks catalogs, grants, clusters, jobs) from declarative, version-controlled configuration files, instead of manual UI clicks.
- **Tokenization —** A pseudonymization method that replaces values with tokens/keys, storing the real values in a secure lookup vault table; slow to write, fast to read.
- **Unity Catalog (UC) —** Databricks' unified governance layer for all data and AI assets (tables, files, models, dashboards, notebooks) across clouds, using one consistent permission and audit model.
- **VACUUM —** A Delta Lake command that permanently deletes old, unreferenced data files no longer needed for time-travel — the step that actually frees storage and finalizes a deletion.
- **Volume —** A Unity Catalog object governing non-tabular (file-based) data such as libraries, configs, and checkpoints; data here cannot be registered as a table.
- **Z-ORDER —** A Delta Lake optimization technique that co-locates related data together in storage, so filter queries can skip irrelevant files and run faster.

Part 9 — Exam-Focused Cheat Sheet

A condensed, last-minute-revision version of everything above. If you only have 10 minutes before the exam, read this section.

9.1 Numbers Worth Memorizing

Fact	Number
GDPR response deadline	30 days
GDPR maximum fine	4% of global annual revenue OR €20 million, whichever is greater
CCPA acknowledgement deadline	10 business days
CCPA resolution deadline	45 days
CCPA fine per violation	$2,500 (up to $7,500 for intentional violations)
Default minimum VACUUM retention	7 days (must be explicitly disabled to go lower)
"Three facts" re-identification rule	As few as 3 combined data points can re-identify a person

9.2 Decision Table: Which Tool for Which Job?

Scenario	Use This
Hide a whole column from most users, no new object	Column Mask (SET MASK)
Show only certain rows to certain groups, no new object	Row Filter (SET ROW FILTER)
Need a separate, reusable filtered/redacted "object" to hand to a BI tool	Dynamic View
Need reversible de-identification for ML work	Pseudonymization (hashing or tokenization)
Need irreversible protection for BI/aggregate reporting	Anonymization (suppression or generalization)
Need to reliably JOIN on a de-identified value	Hashing (deterministic) with a salt
Need fast reads and don't mind slower writes, with true reversibility	Tokenization
Only care about new rows arriving, can ignore changes entirely	ignoreDeletes / skipChangeCommits
Need full row-level change history (inserts/updates/deletes) for ETL, BI, or audit	Change Data Feed (CDF)
Need to permanently delete a user's data end-to-end for GDPR	DELETE/UPDATE on the streaming table + CDF propagation + VACUUM
Need to remove deleted data from a Gold-layer aggregation	Trigger/await a Materialized View refresh
Need to manage grants/infrastructure as version-controlled code	Terraform
Need a quick one-off permission change via UI	Catalog Explorer

Scenario	Use This
Need to script permission changes from the command line	Databricks CLI

9.3 Comparison Recap Tables

Pseudonymization vs. Anonymization

	Pseudonymization	Anonymization
Level	Record-level	Whole dataset
Reversible	Yes (authorized parties)	No
Still PII under GDPR?	Yes	No
Methods	Hashing, Tokenization	Suppression, Generalization

CDF vs. CDC

	CDF	CDC
What it is	Delta Lake's native change-tracking feature	The general concept of capturing source changes
How enabled	TBLPROPERTIES delta.enableChangeDataFeed = true	APPLY CHANGES INTO (in Lakeflow SDP)
Granularity	Row-level, with pre/postimage	Depends on implementation

Streaming Table vs. Materialized View — DML Support

	Streaming Table	Materialized View
Direct INSERT/UPDATE/DELETE/MERGE	Yes	No
How it updates	DML statements	Refresh only (auto in pipelines, or manual REFRESH)

9.4 Key SQL / Config Snippets

```
-- Grant / Revoke
GRANT SELECT ON TABLE t TO analysts;
REVOKE SELECT ON TABLE t FROM analysts;

-- Row filter & column mask
ALTER TABLE t SET ROW FILTER my_filter_fn ON (col1);
ALTER TABLE t ALTER COLUMN ssn SET MASK ssn_mask_fn;

-- Enable CDF
ALTER TABLE t SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

-- Read CDF (batch)
```

```
SELECT * FROM table_changes('cat.schema.t', 5, 10);

-- Read CDF (stream) -- via readStream, source is the table itself
spark.readStream.format('delta').option('readChangeFeed','true').table('cat.schema.t')

-- Ignore changes in a stream
spark.readStream.format('delta').option('skipChangeCommits','true').table('cat.schema.t')

-- Force-vacuum for compliance deletion
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM cat.schema.t RETAIN 0 HOURS DRY RUN;
VACUUM cat.schema.t RETAIN 0 HOURS;

-- DML directly on a streaming table
DELETE FROM cat.schema.users WHERE updated < current_time() - INTERVAL 3 YEARS;
UPDATE cat.schema.users SET email = hash(email, salt) WHERE user_id = 42;
```

9.5 Best-Practice One-Liners

- Prefer groups over individual users for every grant.
- Anonymization > Pseudonymization > Cleartext — always in that order of preference.
- Row Filters/Column Masks avoid duplicate view objects; Dynamic Views are more flexible but need per-case maintenance.
- Migrate off Hive Metastore and DBFS; move models and unstructured files into UC/Volumes.
- Deleted data isn't really gone until VACUUM runs — history/CDF retain old versions until then.
- A Materialized View can never be DML'd directly — only refreshed.
- Isolate the environments that touch PII; it simplifies both security and audits.

Part 10 — Self-Check Practice Questions

15 scenario-style questions in the spirit of the real exam. Try to answer all of them before checking the answer key at the end of this section.

Q1. A company receives a formal data subject access/deletion request under GDPR. Within how many days must they generally respond?

A. 10 business days
B. 24 hours
C. 30 days
D. 90 days

Q2. Which statement about CCPA is correct?

A. It requires acknowledgement of a request within 10 business days and resolution within 45 days
B. It only applies to healthcare companies
C. It has no financial penalties, only warnings
D. It requires a response within 30 days, identical to GDPR

Q3. A team wants to mask an SSN column directly on the base table, without creating any new view objects. What should they use?

A. A Dynamic View
B. A Column Mask (SET MASK)
C. The table_changes() function
D. VACUUM

Q4. What is Terraform, in the context of this chapter?

A. A Databricks-specific SQL dialect for writing ETL jobs
B. An open-source Infrastructure-as-Code tool that manages resources (including Databricks objects) from declarative config files
C. A Delta Lake command for compacting small files
D. A machine learning framework used for PII detection

Q5. Why is a random "salt" added to a value before hashing it for pseudonymization?

A. To make the hash reversible by anyone
B. To reduce the storage size of the resulting hash
C. To defend against dictionary/rainbow-table attacks on the deterministic hash
D. To convert the hash into a token

Q6. Which of the following best describes Tokenization?

A. Fast to write, slow to read, and increases storage size
B. Slow to write, fast to read, and typically decreases storage size, using a secure lookup vault
C. An irreversible technique used only for BI dashboards
D. A technique that only works on numeric columns

Q7. Turning "Amsterdam" into "The Netherlands" in a report is an example of which anonymization method?

A. Data suppression
B. Tokenization

C. Categorical generalization
D. Row filtering

Q8. What is the correct order of Unity Catalog's namespace hierarchy, from top to bottom?

A. Catalog → Metastore → Schema → Table
B. Metastore → Catalog → Schema → Table/View/Volume
C. Schema → Catalog → Metastore → Table
D. Workspace → Metastore → Table → Schema

Q9. Which object is described as the primary unit of data isolation in Unity Catalog?

A. Metastore
B. Workspace
C. Catalog
D. Cluster

Q10. Which system schema would you query to find out which 10 users consumed the most DBUs this month?

A. information_schema
B. billing
C. access
D. default

Q11. Which statement correctly distinguishes CDF from CDC?

A. CDF is the general concept; CDC is Databricks' specific implementation
B. CDC is specific to Delta Lake; CDF works across all database systems
C. CDF is Delta Lake's native, specific implementation of the general CDC concept
D. They are unrelated concepts with no overlap

Q12. A stream should return only inserted rows and completely disregard both updates and deletes to the source Delta table. Which option should be used?

A. ignoreDeletes only
B. skipChangeCommits
C. enableChangeDataFeed
D. readChangeFeed

Q13. What is the correct syntax to read CDF changes between Delta version 5 and version 10 for table sales.gold.orders?

A. SELECT * FROM change_feed('sales.gold.orders', 5, 10)
B. SELECT * FROM table_changes('sales.gold.orders', 5, 10)
C. SELECT * FROM sales.gold.orders CHANGES(5,10)
D. VACUUM sales.gold.orders VERSIONS 5 TO 10

Q14. By default, Delta Lake's VACUUM command will refuse to run with a retention period below how many days, and why?

A. 1 day — to save compute cost
B. 7 days — to protect time-travel/history and prevent accidental data loss
C. 30 days — to match GDPR's response window
D. There is no default protection; VACUUM always runs immediately

Q15. A compliance team needs to permanently remove a user's data from a Gold-layer object that is a Materialized View. What is the correct approach?

A. Run DELETE FROM directly on the materialized view
B. Run MERGE INTO directly on the materialized view
C. Delete the data upstream and trigger/await a refresh of the materialized view
D. Materialized Views cannot be affected by deletions under any circumstance

Answer Key

Q#	Answer	Why
1	C — 30 days	GDPR's standard response window
2	A	CCPA: acknowledge in 10 business days, resolve within 45
3	B — Column Mask	SET MASK attaches directly to the base table; no new view needed
4	B	Terraform = Infrastructure-as-Code tool using declarative config files
5	C	Salting defends against dictionary/rainbow-table reversal of deterministic hashes
6	B	Tokenization: slow write / fast read / smaller storage via a secure vault
7	C — Categorical generalization	Moving from a specific place to a broader category
8	B	Metastore → Catalog → Schema → Table/View/Volume/Model/Function
9	C — Catalog	The metastore gives regional isolation only; the catalog is the real isolation unit
10	B — billing	The billing.usage table answers DBU consumption questions
11	C	CDF is Delta Lake's specific, native implementation of the general CDC concept
12	B — skipChangeCommits	Discards ALL file-changing ops (updates + deletes); supersedes ignoreDeletes
13	B	table_changes(table_str, start [, end]) is the correct syntax
14	B — 7 days	Default retentionDurationCheck protects against premature data loss
15	C	MVs support no direct DML — only refresh (auto in pipelines or manual REFRESH)

Part 11 — Key Resources (From Your Original Slides)

Your source deck ends with a curated resource list for deeper reading. The slide itself only showed link titles (it was pasted in as a screenshot), so the links below were looked up and verified separately — click straight through:

Databricks Documentation

- Monitor account activity with system tables (System tables reference)
- Information schema
- Audit log system table reference
- Lineage system tables reference (covers both table_lineage and column_lineage)
- Unity Catalog best practices
- Security & Trust Center — Data Protection With Customer-Managed Keys

Research Papers on Anonymization & Re-identification

- "How To Break Anonymity of the Netflix Prize Dataset" — arXiv cs/0610105, Narayanan & Shmatikov
- "10 Misunderstandings Related To Anonymisation" (PDF) — joint AEPD/EDPS paper
- "Robust De-anonymization of Large Sparse Datasets" (PDF) — Narayanan & Shmatikov, IEEE S&P 2008
- "De-anonymization of Netflix Reviews using Amazon Reviews" (PDF) — MIT 6.857 course project

> **Why These Matter**
>
> The Netflix Prize and "robust de-anonymization" papers are the real-world proof behind the "three facts rule" and "dataset combination risk" best practices in Part 5.4 — researchers were able to re-identify supposedly anonymous Netflix reviewers just by cross-referencing public IMDb and Amazon reviews. It's worth skimming at least the abstract of these if you want the intuition to stick.